Time Complexity, Space Complexity & Auxiliary Space Complexity in Java (DSA)

Simple Explanation + Real Examples + Quick Summary

1. Time Complexity

What is it?

Time Complexity batati hai ki algorithm ko run karne mein kitna **time** lagega, **input size (n)** badne par.

Example: Linear Search

```
for (int i = 0; i < n; i++) {
    if (arr[i] == x) return i;
}
```

Time: $O(n)$

(har element check karna padega)

Common Notations

- **O (1)** – Constant Time
- **O (log n)** – Logarithmic Time
- **O (n)** – Linear Time
- **O (n log n)** – Linearithmic Time
- **O (n^2)** – Quadratic Time
- **O (2^n)** – Exponential Time

Real Life Analogy

Jaise n students ko ek-ek karke check karna – to time O(n) lagega.



2. Space Complexity

What is it?

Space Complexity batati hai ki algorithm ko run karne ke liye kitni **memory** (extra space) chahiye.

Example: Array Sum

```
int sum = 0;
for (int x : arr) {
    sum += x;
}
```

Space: O(1)

(sirf ek variable sum)

Common Notations

- **O (1)** – Constant Space
- **O (n)** – Linear Space
- **O (n^2)** – Quadratic Space

Real Life Analogy

Jaise ek **counter** (rakha hua ek variable) hi enough hai – to space O(1) hoga.



3. Auxiliary Space Complexity

What is it?

Auxiliary Space Complexity batati hai ki algorithm ko **extra space** (input ke alawa) kitni memory chahiye.

Example: Recursive Fibonacci

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
```

Auxiliary Space: $O(n)$

(function call stack ki wajah se)

Key Difference

- **Space Complexity: Total memory (input + extra).**
- **Auxiliary Space: Sirf extra memory (input ke alawa).**

Real Life Analogy

Jaise kisi kaam ke liye extra staff rakhna (puri team nahi, bas extra log) – ye auxiliary space hai.



Quick Summary Table

Topic	Kya Batata Hai?	Notation	Example
Time Complexity	Kitna time lagega	O(n), O(n^2), O(log n)	Linear Search → O(n)
Space Complexity	Kitni memory lagegi (total)	O(1), O(n)	Array Sum → O(1)
Auxiliary Space Complexity	Kitni extra memory lagegi (input ke alawa)	O(n), O(1)	Recursive Fibonacci → O(n)

Key Takeaways

- **Time Complexity** = Speed of Algorithm
- **Space Complexity** = Memory Usage
- **Auxiliary Space** = Extra Memory Only
- Use these to analyze and **compare** algorithms in **DSA + Interviews!**

Better Analysis = Better Coding + Better Performance!

Time Complexity, Space Complexity & Auxiliary Space Complexity in Java (DSA)

Complete Concept + Examples + Important Topics

1. Time Complexity
(How fast algorithm runs?)

- It tells us how the running time grows with input size n.
- **Common Units:** O(1), O(log n), O(n), O(n log n), O(n^2), O(2^n)

Example: Linear Search

```
for (int i = 0; i < n; i++) {
    if (arr[i] == x) return i;
}
```

→ **Time: O(n)**

2. Space Complexity
(How much memory needed?)

- It tells us how much extra memory our algorithm uses.
- Includes input storage + extra variables, arrays, etc.

Example: Array Sum

```
int sum = 0;
for (int x : arr) sum += x;
```

→ **Space: O(1)**

3. Auxiliary Space Complexity
(Extra memory used by algorithm)

- It is the additional space (besides input) used by the algorithm.
- Includes variables, data structures, recursion stack, etc.

Example: Recursive Fibonacci

```
if (n <= 1) return n;
return fib(n-1) + fib(n-2);
```

→ **Auxiliary Space: O(n)** (recursion stack)

4. Common Notations & Their Meaning

O(1)	O(log n)	O(n)	O(n log n)	O(n^2)	O(2^n)
Constant (e.g. return 1)	**Logarithmic** (e.g. binary search)	**Linear** (e.g. traversal)	**Linearithmic** (e.g. merge sort)	**Quadratic** (e.g. nested loop)	**Exponential** (e.g. subsets)

5. Important Topics with Examples

A. Arrays & Strings
Sum of array:
`for (int x : arr) sum += x;`
→ O(n), Space: O(1)

B. Searching & Sorting
Binary Search:
`while (l <= r) { ... }`
→ O(log n), Space: O(1)

C. Recursion
Factorial:
`return n * fact(n-1);`
→ O(n), Aux Space: O(n)

D. Divide & Conquer
Merge Sort:
divide + merge
→ O(n log n), Space: O(n)

E. Dynamic Programming
Fibonacci DP:
`dp[i] = dp[i-1] + dp[i-2];`
→ O(n), Space: O(n)

F. Graph Algorithms
BFS:
Queue + visit[]
→ O(V+E), Space: O(V)

G. Heaps & Priority Queue
Heap Sort:
`PriorityQueue<Integer>`
→ O(n log n), Space: O(1)

H. Hashing
HashMap Lookup:
`map.get(key);`
→ O(1), Space: O(n)

I. Stacks & Queues
Stack Push/Pop:
`push(), pop()`
→ O(1), Space: O(n)

J. Trees
DFS on Tree:
`dfs(node);`
→ O(n), Space: O(h)

6. More Examples (Detailed)

1. Nested Loops

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        System.out.println(i + j);
```

→ Time: O(n^2), Space: O(1)

2. ArrayList Add

```
list.add(x);
```

→ Amortized O(1), Space: O(n)

3. DFS (Recursive)

```
void dfs(int u){
    vis[u] = true;
    for(int v : adj[u])
        dfs(v);
}
```

→ O(V+E), Space: O(V)

4. DP – 0/1 Knapsack

```
for (int i = 1; i <= n; i++)
    for (int w = 0; w <= W; w++)
        dp[i][w] = ...;
```

→ O(nW), Space: O(nW)

5. Backtracking (N-Queens)

```
placeQueen(row) {...}
```

→ O(nW), Space: O(N)

7. Quick Comparison Table

Topic	Time Complexity	Space Complexity	Auxiliary Space
Linear Search	O(n)	O(1)	O(1)
Binary Search	O(log n)	O(1)	O(1)
Merge Sort	O(n log n)	O(n)	O(n)
Fibonacci (Recursion)	O(2^n)	O(n)	O(n)
BFS / DFS	O(V + E)	O(V)	O(V)
Heap Sort	O(n log n)	O(1)	O(1)

8. Key Takeaways

- ✅ **Time Complexity** = Speed (Big-O Notation).
- ✅ **Space Complexity** = Memory Usage.
- ✅ **Auxiliary Space** = Extra Memory (beyond input).
- ✅ **Common Orders:** 1, log n, n, n log n, n^2, 2^n.
- ✅ Always analyze **both time** & space for a good algorithm.

9. Useful Tips

- ★ Brute Force usually O(n^2).
- ★ Binary Search is O(log n).
- ★ Recursion adds space (stack).
- ★ DP trades space for speed.
- ★ Use Hashing for O(1) lookup.
- ★ Prefer better complexity for large inputs!